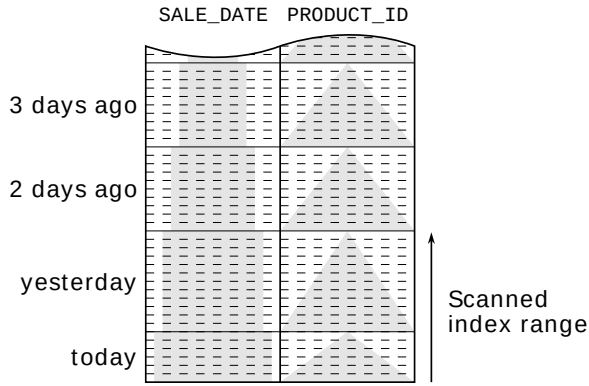
Figure 6.2. Reverse Index Scan

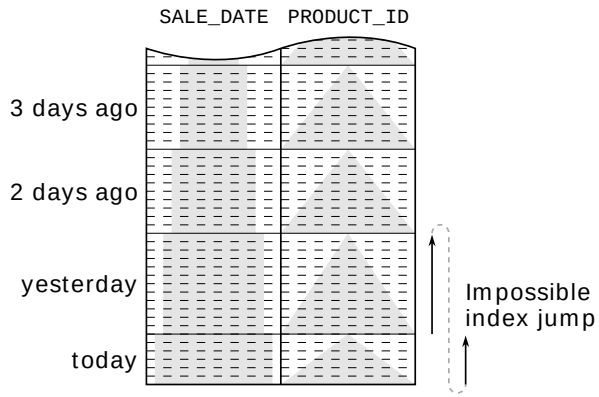


The following example does not fulfill this prerequisite because it mixes ASC and DESC modifiers in the **order by** clause:

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date >= TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY sale_date ASC, product_id DESC;
```

The query must first deliver yesterday's sales ordered by descending PRODUCT_ID and then today's sales, again by descending PRODUCT_ID. Figure 6.3 illustrates this process. To get the sales in the required order, the database would have to "jump" during the index scan.

Figure 6.3. Impossible Pipelined order by



However, the index has no link from yesterday's sale with the smallest PRODUCT_ID to today's sale with the greatest. The database can therefore not use this index to avoid an explicit sort operation.

For cases like this, most databases offer a simple method to adjust the index order to the **order by** clause. Concretely, this means that you can use ASC and DESC modifiers in the index declaration:

```
  DROP INDEX sales_dt_pr;

CREATE INDEX sales_dt_pr
    ON sales (sale_date ASC, product_id DESC);
```



Warning

The MySQL database ignores ASC and DESC modifiers in the index definition.

Now the index order corresponds to the **order by** clause so the database can omit the sort operation:

```
-------------------------------------------------------------------
|Id | Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |             |  320 |  301 |
| 1 |  TABLE ACCESS BY INDEX ROWID| SALES       |  320 |  301 |
|*2 |   INDEX RANGE SCAN           | SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```

Figure 6.4 shows the new index order. The change in the sort direction for the second column in a way swaps the direction of the arrows from the previous figure. That makes the first arrow end where the second arrow starts so that index has the rows in the desired order.



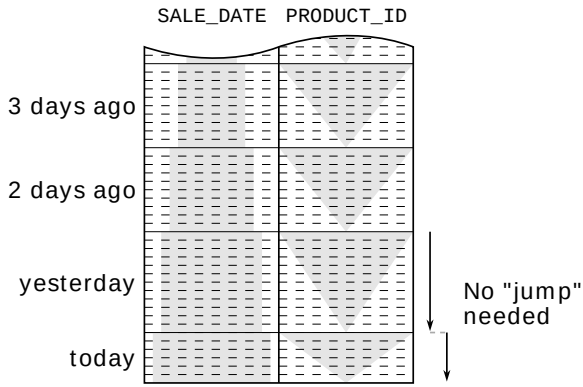
Important

When using mixed ASC and DESC modifiers in the **order by** clause, you must define the index likewise in order to use it for a pipelined **order by**.

This does not affect the index's usability for the **where** clause.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Figure 6.4. Mixed-Order Index



ASC/DESC indexing is only needed for sorting individual columns in opposite direction. It is not needed to reverse the order of all columns because the database could still read the index in descending order if needed—secondary indexes on index organized tables being the only exception. Secondary indexes implicitly add the clustering key to the index without providing any possibility for specifying the sort order. If you need to sort the clustering key in descending order, you have no other option than sorting all other columns in descending order. The database can then read the index in reverse direction to get the desired order.

Besides ASC and DESC, the SQL standard defines two hardly known modifiers for the **order by** clause: NULLS FIRST and NULLS LAST. Explicit control over NULL sorting was "recently" introduced as an *optional* extension with SQL:2003. As a consequence, database support is sparse. This is particularly worrying because the standard does not exactly define the sort order of NULL. It only states that all NULLs must appear together after sorting, but it does not specify if they should appear before or after the other entries. Strictly speaking, you would actually need to specify NULL sorting for all columns that can be null in the **order by** clause to get a well-defined behavior.

The fact is, however, that the optional extension is neither implemented by SQL Server 2012 nor by MySQL 5.6. The Oracle database, on the contrary, supported NULLS sorting even before it was introduced to the standard, but it does not accept it in index definitions as of release 11*g*. The Oracle database can therefore not do a pipelined **order by** when sorting with

NULLS FIRST. Only the PostgreSQL database (since release 8.3) supports the NULLS modifier in both the **order by** clause and the index definition.

The following overview summarizes the features provided by different databases.

Figure 6.5. Database/Feature Matrix

	MySQL	Oracle	PostgreSQL	SQL Server
Read index backwards	✓	✓	✓	✓
Order by ASC/DESC	✓	✓	✓	✓
Index ASC/DESC	✗	✓	✓	✓
Order by NULLS FIRST/LAST	✗	✓	✓	✗
Default NULLS order	First	Last	Last	First
Index NULLS FIRST/LAST	✗	✗	✓	✗

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Indexing Group By

SQL databases use two entirely different **group by** algorithms. The first one, the hash algorithm, aggregates the input records in a temporary hash table. Once all input records are processed, the hash table is returned as the result. The second algorithm, the sort/group algorithm, first sorts the input data by the grouping key so that the rows of each group follow each other in immediate succession. Afterwards, the database just needs to aggregate them. In general, both algorithms need to materialize an intermediate state, so they are not executed in a pipelined manner. Nevertheless the sort/group algorithm can use an index to avoid the sort operation, thus enabling a pipelined **group by**.



Note

MySQL 5.6 doesn't use the hash algorithm. Nevertheless, the optimization for the sort/group algorithm works as described below.

Consider the following query. It delivers yesterday's revenue grouped by PRODUCT_ID:

```
SELECT product_id, sum(eur_value)
  FROM sales
 WHERE sale_date = TRUNC(sysdate) - INTERVAL '1' DAY
 GROUP BY product_id;
```

Knowing the index on SALE_DATE and PRODUCT_ID from the previous section, the sort/group algorithm is more appropriate because an INDEX RANGE SCAN automatically delivers the rows in the required order. That means the database avoids materialization because it does not need an explicit sort operation—the **group by** is executed in a pipelined manner.

```
-------------------------------------------------------------------
|Id |Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT              |             |   17 |  192 |
| 1 | SORT GROUP BY NOSORT         |             |   17 |  192 |
| 2 |  TABLE ACCESS BY INDEX ROWID| SALES       |  321 |  192 |
|*3 |   INDEX RANGE SCAN           | SALES_DT_PR |  321 |    3 |
-------------------------------------------------------------------
```

The Oracle database's execution plan marks a pipelined SORT GROUP BY operation with the NOSORT addendum. The execution plan of other databases does not mention any sort operation at all.

The pipelined **group by** has the same prerequisites as the pipelined **order by**, except there are no ASC and DESC modifiers. That means that defining an index with ASC/DESC modifiers should not affect pipelined **group by** execution. The same is true for NULLS FIRST/LAST. Nevertheless there are databases that cannot properly use an ASC/DESC index for a pipelined **group by**.



Warning

For PostgreSQL, you must add an **order by** clause to make an index with NULLS LAST sorting usable for a pipelined **group by**.

The Oracle database cannot read an index backwards in order to execute a pipelined **group by** that is followed by an **order by**.

If we extend the query to consider all sales *since yesterday*, as we did in the example for the pipelined **order by**, it prevents the pipelined **group by** for the same reason as before: the INDEX RANGE SCAN does not deliver the rows ordered by the grouping key (compare Figure 6.1 on page 131).

```
SELECT product_id, sum(eur_value)
  FROM sales
 WHERE sale_date >= TRUNC(sysdate) - INTERVAL '1' DAY
 GROUP BY product_id;

-------------------------------------------------------------------
|Id |Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT              |             |   24 |  356 |
| 1 | HASH GROUP BY                |             |   24 |  356 |
| 2 |  TABLE ACCESS BY INDEX ROWID| SALES       |  596 |  355 |
|*3 |   INDEX RANGE SCAN           | SALES_DT_PR |  596 |    4 |
-------------------------------------------------------------------
```

Instead, the Oracle database uses the hash algorithm. The advantage of the hash algorithm is that it only needs to buffer the *aggregated result*, whereas the sort/group algorithm materializes the *complete input set*. In other words: the hash algorithm needs less memory.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

As with pipelined **order by**, a fast execution is not the most important aspect of the pipelined **group by** execution. It is more important that the database executes it in a pipelined manner and delivers the first result before reading the entire input. This is the prerequisite for the advanced optimization methods explained in the next chapter.



Think about it

Can you think of any other database operation—besides sorting and grouping—that could possibly use an index to avoid sorting?

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 7

Partial Results

Sometimes you do not need the full result of an SQL query but only the first few rows—e.g., to show only the ten most recent messages. In this case, it is also common to allow users to browse through older messages—either using traditional paging navigation or the more modern "infinite scrolling" variant. The related SQL queries used for this function can, however, cause serious performance problems if *all* messages must be sorted in order to find the most recent ones. A pipelined **`order by`** is therefore a very powerful means of optimization for such queries.

This chapter demonstrates how to use a pipelined **`order by`** to efficiently retrieve partial results. Although the syntax of these queries varies from database to database, they still execute the queries in a very similar way. Once again, this illustrates that they all put their pants on one leg at a time.

Querying Top-N Rows

Top-N queries are queries that limit the result to a specific number of rows. These are often queries for the most recent or the "best" entries of a result set. For efficient execution, the ranking must be done with a pipelined **`order by`**.

The simplest way to fetch only the first rows of a query is fetching the required rows and then closing the statement. Unfortunately, the optimizer cannot foresee that when preparing the execution plan. To select the best execution plan, the optimizer has to know if the application will ultimately fetch all rows. In that case, a full table scan with explicit sort operation might perform best, although a pipelined **`order by`** could be better when fetching only ten rows—even if the database has to fetch each row individually. That means that the optimizer has to know if you are going to abort the statement before fetching all rows so it can select the best execution plan.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>



Tip

Inform the database whenever you don't need all rows.

The SQL standard excluded this requirement for a long time. The corresponding extension (**`fetch first`**) was just introduced with SQL:2008 and is currently only available in IBM DB2, PostgreSQL and SQL Server 2012. On the one hand, this is because the feature is a non-core extension, and on the other hand it's because each database has been offering its own proprietary solution for many years.

The following examples show the use of these well-known extensions by querying the ten most recent sales. The basis is always the same: fetching *all* sales, beginning with the most recent one. The respective top-N syntax just aborts the execution after fetching ten rows.

MySQL
: MySQL and PostgreSQL use the **`limit`** clause to restrict the number of rows to be fetched.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
 LIMIT 10;
```

Oracle Database
: The Oracle database provides the pseudo column ROWNUM that numbers the rows in the result set automatically. To use this column in a filter, we have to wrap the query:

```
SELECT *
  FROM (
       SELECT *
         FROM sales
        ORDER BY sale_date DESC
       )
 WHERE rownum <= 10;
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

PostgreSQL
: PostgreSQL supports the **`fetch first`** extension since version 8.4. The previously used **`limit`** clause still works as shown in the MySQL example.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
 FETCH FIRST 10 ROWS ONLY;
```

SQL Server
: SQL Server provides the **`top`** clause to restrict the number of rows to be fetched.

```
SELECT TOP 10 *
  FROM sales
 ORDER BY sale_date DESC;
```

  Starting with release 2012, SQL Server supports the **`fetch first`** extension as well.

All of the above shown SQL queries are special because the databases recognize them as top-N queries.



Important

The database can only optimize a query for a partial result if it knows this from the beginning.

If the optimizer is aware of the fact that we only need ten rows, it will prefer to use a pipelined **`order by`** if applicable:

```
-------------------------------------------------------------
| Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------
| SELECT STATEMENT              |             |   10 |    9 |
|  COUNT STOPKEY                |             |      |      |
|   VIEW                        |             |   10 |    9 |
|    TABLE ACCESS BY INDEX ROWID| SALES       | 1004K|    9 |
|     INDEX FULL SCAN DESCENDING| SALES_DT_PR |   10 |    3 |
-------------------------------------------------------------
```

The Oracle execution plan indicates the planned termination with the `COUNT STOPKEY` operation. That means the database recognized the top-N syntax.



TIP

Appendix A, "*Execution Plans*", summarizes the corresponding operations for MySQL, Oracle, PostgreSQL and SQL Server.

Using the correct syntax is only half the story because efficiently terminating the execution requires the underlying operations to be executed in a pipelined manner. That means the **`order by`** clause must be covered by an index—the index `SALE_DT_PR` on `SALE_DATE` and `PRODUCT_ID` in this example. By using this index, the database can avoid an explicit sort operation and so can immediately send the rows to the application as read from the index. The execution is aborted after fetching ten rows so the database does not read more rows than selected.



IMPORTANT

A pipelined top-N query doesn't need to read and sort the entire result set.

If there is no suitable index on `SALE_DATE` for a pipelined **`order by`**, the database must read and sort the entire table. The first row is only delivered after reading the last row from the table.

```
-------------------------------------------------
| Operation               | Name  | Rows |  Cost |
-------------------------------------------------
| SELECT STATEMENT        |       |   10 | 59558 |
|  COUNT STOPKEY          |       |      |       |
|   VIEW                  |       | 1004K| 59558 |
|    SORT ORDER BY STOPKEY|       | 1004K| 59558 |
|     TABLE ACCESS FULL   | SALES | 1004K|  9246 |
-------------------------------------------------
```

This execution plan has no pipelined **`order by`** and is almost as slow as aborting the execution from the client side. Using the top-N syntax is still better because the database does not need to materialize the full result but only the ten most recent rows. This requires considerably less memory. The Oracle execution plan indicates this optimization with the `STOPKEY` modifier on the `SORT ORDER BY` operation.

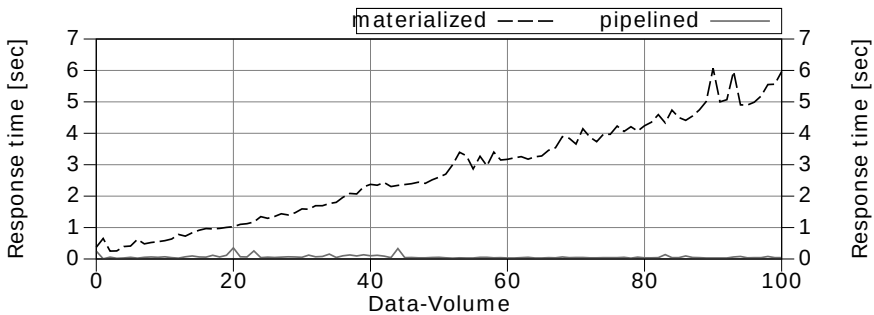
The advantages of a pipelined top-N query include not only immediate performance gains but also improved scalability. Without using pipelined execution, the response time of this top-N query grows with the table size. The response time using a pipelined execution, however, only grows

EX LIBRIS GHEORGHE GABRIEL SICHIM <GABRIEL.SICHIM@GMAIL.COM>

with the number of selected rows. In other words, the response time of a pipelined top-N query is always the same; this is almost independent of the table size. Only when the B-tree depth grows does the query become a little bit slower.

Figure 7.1 shows the scalability for both variants over a growing volume of data. The linear response time growth for an execution without a pipelined **order by** is clearly visible. The response time for the pipelined execution remains constant.

Figure 7.1. Scalability of Top-N Queries



Although the response time of a pipelined top-N query does not depend on the table size, it still grows with the number of selected rows. The response time will therefore double when selecting twice as many rows. This is particularly significant for "paging" queries that load additional results because these queries often start at the first entry again; they will read the rows already shown on the previous page and discard them before finally reaching the results for the second page. Nevertheless, there is a solution for this problem as well as we will see in the next section.

Paging Through Results

After implementing a pipelined top-N query to retrieve the first page efficiently, you will often also need another query to fetch the next pages. The resulting challenge is that it has to skip the rows from the previous pages. There are two different methods to meet this challenge: firstly the *offset method*, which numbers the rows from the beginning and uses a filter on this row number to discard the rows before the requested page. The second method, which I call the *seek method*, searches the last entry of the previous page and fetches only the following rows.

The following examples show the more widely used offset method. Its main advantage is that it is very easy to handle—especially with databases that have a dedicated keyword for it (**offset**). This keyword was even taken into the SQL standard as part of the **fetch first** extension.

MySQL
: MySQL and PostgreSQL offer the **offset** clause for discarding the specified number of rows from the beginning of a top-N query. The **limit** clause is applied afterwards.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
 LIMIT 10 OFFSET 10;
```

Oracle Database
: The Oracle database provides the pseudo column ROWNUM that numbers the rows in the result set automatically. It is, however, not possible to apply a greater than or equal to (>=) filter on this pseudo-column. To make this work, you need to first "materialize" the row numbers by renaming the column with an alias.

```
SELECT *
  FROM ( SELECT tmp.*, rownum rn
           FROM ( SELECT *
                    FROM sales
                   ORDER BY sale_date DESC
                ) tmp
          WHERE rownum <= 20
       )
 WHERE rn > 10;
```

  Note the use of the alias RN for the lower bound and the ROWNUM pseudo column itself for the upper bound.

PostgreSQL
: The **fetch first** extension defines an **offset ... rows** clause as well. PostgreSQL, however, only accepts **offset** without the **rows** keyword. The previously used **limit/offset** syntax still works as shown in the MySQL example.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
OFFSET 10
 FETCH NEXT 10 ROWS ONLY;
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

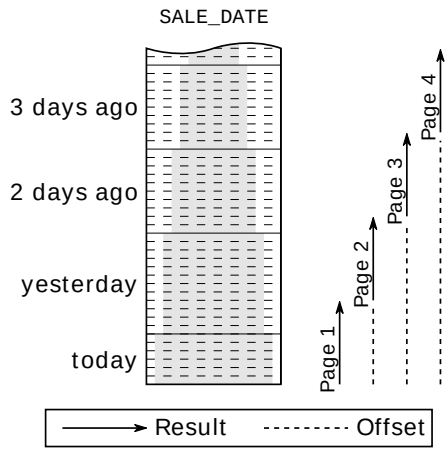
SQL Server

SQL Server does not have an "offset" extension for its proprietary **top** clause but introduced the **fetch first** extension with SQL Server 2012 ("Denali"). The **offset** clause is mandatory although the standard defines it as an optional addendum.

```
SELECT *
  FROM sales
 ORDER BY sale_date DESC
OFFSET 10 ROWS
 FETCH NEXT 10 ROWS ONLY;
```

Besides the simplicity, another advantage of this method is that you just need the row offset to fetch an arbitrary page. Nevertheless, the database must count all rows from the beginning until it reaches the requested page. Figure 7.2 shows that the scanned index range becomes greater when fetching more pages.

Figure 7.2. Access Using the Offset Method



This has two disadvantages: (1) the pages drift when inserting new sales because the numbering is always done from scratch; (2) the response time increases when browsing further back.

The seek method avoids both problems because it uses the *values* of the previous page as a delimiter. That means it searches for the values that must *come behind* the last entry from the previous page. This can be expressed with a simple **where** clause. To put it the other way around: the seek method simply doesn't select already shown values.

The next example shows the seek method. For the sake of demonstration, we will start with the assumption that there is only one sale per day. This makes the SALE_DATE a unique key. To select the sales that must come behind a particular date you must use a less than condition (<) because of the descending sort order. For an ascending order, you would have to use a greater than (>) condition. The **fetch first** clause is just used to limit the result to ten rows.

```
SELECT *
  FROM sales
 WHERE sale_date < ?
 ORDER BY sale_date DESC
 FETCH FIRST 10 ROWS ONLY;
```

Instead of a row number, you use the last value of the previous page to specify the lower bound. This has a huge benefit in terms of performance because the database can use the SALE_DATE < ? condition for index access. That means that the database can truly skip the rows from the previous pages. On top of that, you will also get stable results if new rows are inserted.

Nevertheless, this method does not work if there is more than one sale per day—as shown in Figure 7.2—because using the last date from the first page ("yesterday") skips *all* results from yesterday—not just the ones already shown on the first page. The problem is that the **order by** clause does not establish a deterministic row sequence. That is, however, prerequisite to using a simple range condition for the page breaks.

Without a deterministic **order by** clause, the database by definition does not deliver a deterministic row sequence. The only reason you *usually* get a consistent row sequence is that the database *usually* executes the query in the same way. Nevertheless, the database could in fact shuffle the rows having the same SALE_DATE and still fulfill the **order by** clause. In recent releases it might indeed happen that you get the result in a different order every time you run the query, not because the database shuffles the result intentionally but because the database might utilize parallel query execution. That means that the same execution plan can result in a different row sequence because the executing threads finish in a non-deterministic order.



Important

Paging requires a deterministic sort order.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Even if the functional specifications only require sorting "by date, latest first", we as the developers must make sure the **order by** clause yields a deterministic row sequence. For this purpose, we might need to extend the **order by** clause with arbitrary columns just to make sure we get a deterministic row sequence. If the index that is used for the pipelined **order by** has additional columns, it is a good start to add them to the **order by** clause so we can continue using this index for the pipelined **order by**. If this still does not yield a deterministic sort order, just add any unique column(s) and extend the index accordingly.

In the following example, we extend the **order by** clause and the index with the primary key SALE_ID to get a deterministic row sequence. Furthermore, we must apply the "comes after" logic to both columns *together* to get the desired result:

```
CREATE INDEX sl_dtid ON sales (sale_date, sale_id);

SELECT *
  FROM sales
 WHERE (sale_date, sale_id) < (?, ?)
 ORDER BY sale_date DESC, sale_id DESC
 FETCH FIRST 10 ROWS ONLY;
```

The **where** clause uses the little-known "row values" syntax (see the box entitled "SQL Row Values"). It combines multiple values into a logical unit that is applicable to the regular comparison operators. As with scalar values, the less-than condition corresponds to "comes after" when sorting in descending order. That means the query considers only the sales that come after the given SALE_DATE, SALE_ID pair.

Even though the row values syntax is part of the SQL standard, only a few databases support it. SQL Server 2012 ("Denali") does not support row values at all. The Oracle database supports row values in principle, but cannot apply range operators on them (ORA-01796). MySQL evaluates row value expressions correctly but cannot use them as access predicate during an index access. PostgreSQL, however, supports the row value syntax *and* uses them to access the index if there is a corresponding index available.

Nevertheless it is possible to use an approximated variant of the seek method with databases that do not properly support the row values—even though the approximation is not as elegant and efficient as row values in PostgreSQL. For this approximation, we must use "regular" comparisons to express the required logic as shown in this Oracle example:

```
SELECT *
  FROM ( SELECT *
           FROM sales
          WHERE sale_date <= ?
            AND NOT (sale_date = ? AND sale_id >= ?)
          ORDER BY sale_date DESC, sale_id DESC
       )
 WHERE rownum <= 10;
```

The **where** clause consists of two parts. The first part considers the SALE_DATE only and uses a less than or equal to (<=) condition—it selects more rows as needed. This part of the **where** clause is simple enough so that all databases can use it to access the index. The second part of the **where** clause removes the excess rows that were already shown on the previous page. The box entitled "Indexing Equivalent Logic" explains why the **where** clause is expressed this way.

The execution plan shows that the database uses the first part of the **where** clause as access predicate.

```
-------------------------------------------------------------------
|Id | Operation                       | Name    |  Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT                |         |    10 |    4 |
|*1 |  COUNT STOPKEY                  |         |       |      |
| 2 |   VIEW                          |         |    10 |    4 |
| 3 |    TABLE ACCESS BY INDEX ROWID | SALES   | 50218 |    4 |
|*4 |     INDEX RANGE SCAN DESCENDING| SL_DTIT |     2 |    3 |
-------------------------------------------------------------------

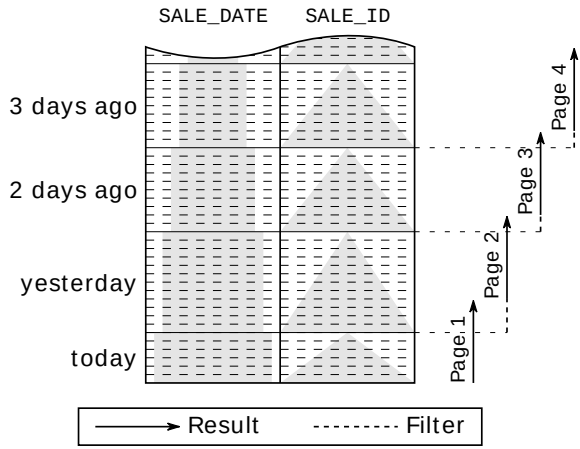
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(ROWNUM<=10)
   4 - access("SALE_DATE"<=:SALE_DATE)
       filter("SALE_DATE"<>:SALE_DATE
           OR "SALE_ID"<TO_NUMBER(:SALE_ID))
```

The access predicates on SALE_DATE enables the database to skip over the days that were fully shown on previous pages. The second part of the **where** clause is a filter predicate only. That means that the database inspects a

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

few entries from the previous page again, but drops them immediately. Figure 7.3 shows the respective access path.

Figure 7.3. Access Using the Seek Method



SQL Row Values

Besides regular scalar values, the SQL standard also defines the so-called *row value constructors*. They "Specify an ordered set of values to be constructed into a row or partial row" [SQL:92, §7.1: <row value constructor>]. Syntactically, row values are lists in brackets. This syntax is best known for its use in the **`insert`** statement.

Using row value constructors in the **`where`** clause is, however, less well-known but still perfectly valid. The SQL standard actually defines all comparison operators for row value constructors. The definition for the less than operations is, for example, as follows:

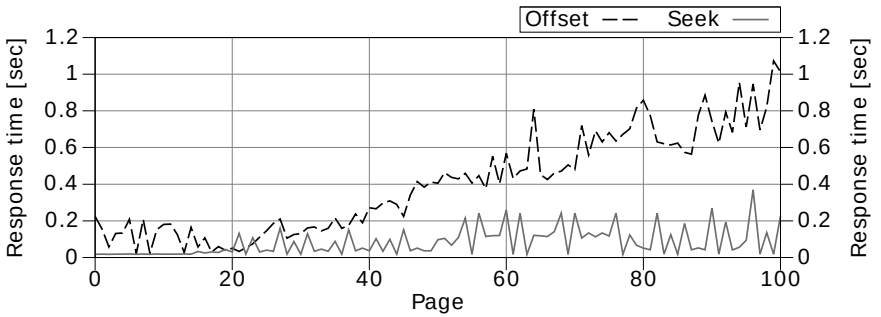
> "Rx < Ry" is true if and only if RXi = RYi for all i < n and RXn < RYn for some n.
>
> —SQL:92, §8.2.7.2

Where *i* and *n* reflect positional indexes in the lists. That means a row value RX is less than RY if any value RXn is smaller than the corresponding RYn and all preceding value pairs are equal (*RXi = RYi; for i<n*).

This definition makes the expression RX < RY synonymous to "RX sorts before RY" which is exactly the logic we need for the seek method.

Figure 7.4 compares the performance characteristics of the offset and the seek methods. The accuracy of measurement is insufficient to see the difference on the left hand side of the chart, however the difference is clearly visible from about page 20 onwards.

Figure 7.4. Scalability when Fetching the Next Page



Of course the seek method has drawbacks as well, the difficulty in handling it being the most important one. You not only have to phrase the `where` clause very carefully—you also cannot fetch arbitrary pages. Moreover you need to reverse all comparison and sort operations to change the browsing direction. Precisely these two functions—skipping pages and browsing backwards—are not needed when using an infinite scrolling mechanism for the user interface.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Indexing Equivalent Logic

A logical condition can always be expressed in different ways. You could, for example, also implement the above shown skip logic as follows:

```
WHERE (
         (sale_date < ?)
       OR
         (sale_date = ? AND sale_id < ?)
      )
```

This variant only uses including conditions and is probably easier to understand—for human beings, at least. Databases have a different point of view. They do not recognize that the **where** clause selects all rows starting with the respective SALE_DATE/SALE_ID pair—provided that the SALE_DATE is the same for both branches. Instead, the database uses the entire **where** clause as filter predicate. We could at least expect the optimizer to "factor the condition SALE_DATE <= ? out" of the two or-branches, but none of the databases provides this service.

Nevertheless we can add this redundant condition manually—even though it does not increase readability:

```
WHERE sale_date <= ?
  AND (
         (sale_date < ?)
       OR
         (sale_date = ? AND sale_id < ?)
      )
```

Luckily, all databases are able to use the this part of the **where** clause as access predicate. That clause is, however, even harder to grasp as the approximation logic shown above. Further, the original logic avoids the risk that the "unnecessary" (redundant) part is accidentally removed from the **where** clause later on.